

Message Relay - HLD

1. High Level Design (HLD)

1.1 Purpose and Scope

Message Relay is a Spring Boot based notification delivery engine. It consumes events from Kafka topics, validates payloads, routes by channel, persists delivery state, indexes searchable records, and handles failures with retries and DLQ.

Primary channels in current scope:

- Email
- SMS
- WhatsApp
- Payment notification stream (processed by same delivery pipeline)

1.2 System Context

External systems interacting with Message Relay:

- Upstream producers publish `NotificationEvent` messages to Kafka topics.
- Message Relay consumes and processes messages.
- MongoDB stores authoritative delivery records and status history.
- Elasticsearch stores denormalized searchable delivery documents.
- Sentry receives DLQ/failure alerts.
- Prometheus scrapes actuator metrics.

1.3 High Level Architecture

Core runtime building blocks:

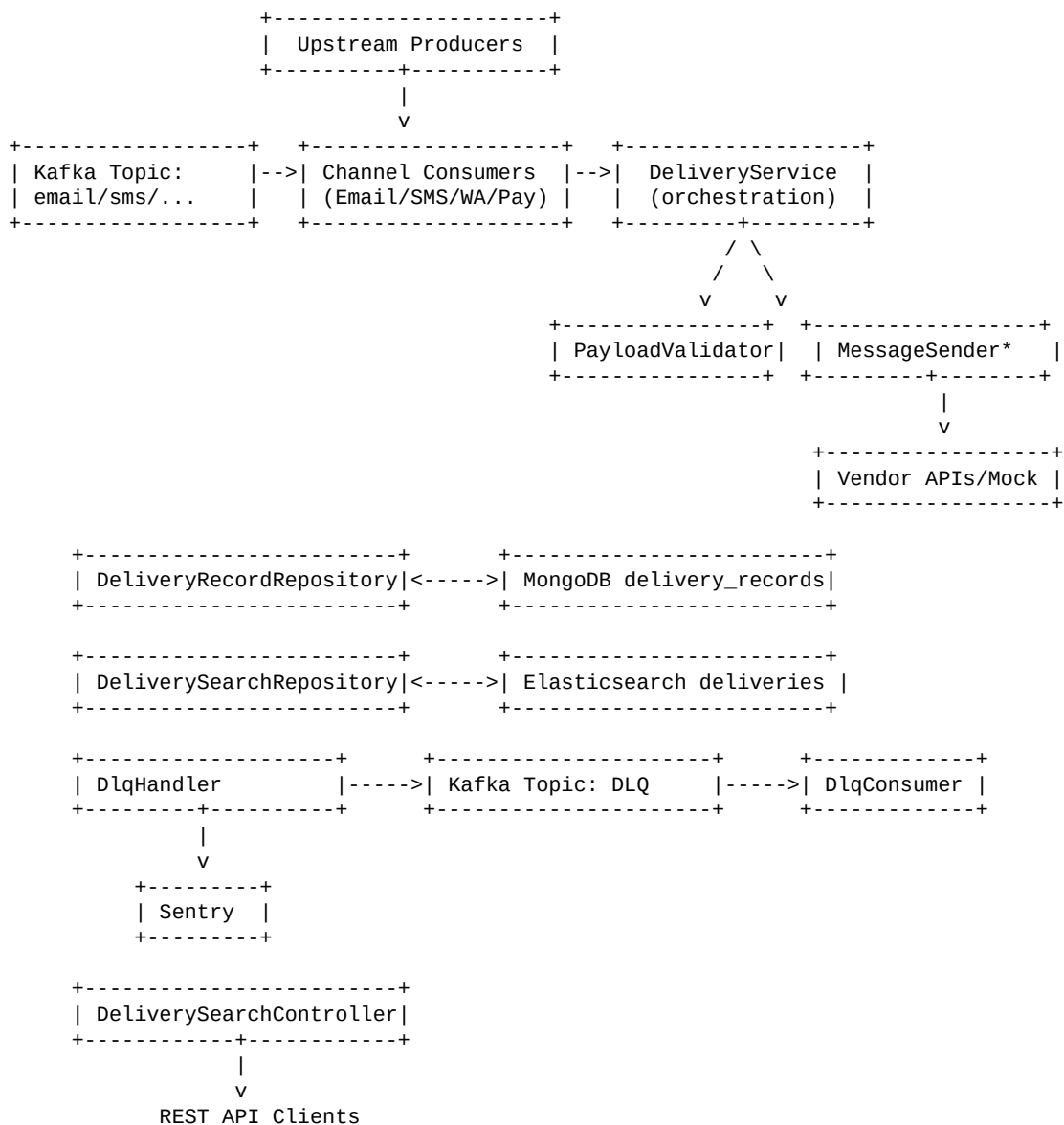
- Kafka consumer layer: channel-specific listeners.
- Delivery orchestration service: validation, persistence, send, retry/circuit-breaker.
- Sender strategy layer: channel implementations selected via factory.
- Persistence layer: MongoDB repository + Elasticsearch repository.
- Failure handling: DLQ publisher + DLQ monitoring consumer.
- API layer: search/status endpoints.
- Observability: health indicators, metrics, logs, Sentry.

1.4 Logical Component Diagram (Textual)

1. Kafka Topics (`email`, `sms`, `whatsapp`, `payment`) -> 2. Consumers
2. Consumers -> `DeliveryService.process(event)`
3. `DeliveryService` -> `PayloadValidator`
4. `DeliveryService` -> `DeliveryRecordRepository` (MongoDB)
5. `DeliveryService` -> `MessageSenderFactory` -> concrete sender
6. Success path -> update MongoDB + index into Elasticsearch
7. Failure path (after retry/circuit-breaker fallback) -> `DlqHandler` -> Kafka DLQ topic + Sentry

8. API Controller -> Elasticsearch (search) + MongoDB (record details, stats)

1.4.1 Component Diagram (ASCII)



1.5 Data Flow (End-to-End)

1. Producer sends `NotificationEvent` to channel topic.
2. Topic listener receives message with manual acknowledgment.
3. Service validates event payload and recipient format.
4. Service creates initial `DeliveryRecord` with status `PENDING`.
5. Service attempts delivery through channel sender with Resilience4j retry + circuit-breaker.
6. On success:
 - Record transitions to `SENT`.
 - Vendor message id is stored.
 - Search document is saved to Elasticsearch.
7. On repeated failure:
 - Record transitions to `DEAD\_LETTERED`.

- Original event is published to DLQ topic.
- Sentry alert is emitted with masked recipient.

1.6 Deployment View

Single service process (Spring Boot) typically deployed as containerized app.

Dependencies:

- Kafka broker reachable at ``spring.kafka.bootstrap-servers``
- MongoDB instance
- Elasticsearch instance
- Optional Sentry DSN configuration
- Prometheus scraping ``/actuator/prometheus``

Default server port: ``8090``.

1.7 Key Non-Functional Characteristics

- Reliability:
 - Retries with exponential backoff (``max-attempts=3``, wait ``2s``, multiplier ``2``).
 - Circuit breaker for sender operations.
 - DLQ pattern for poison or persistently failing messages.
- Scalability:
 - Kafka listener concurrency configured to 3 for main listeners.
- Observability:
 - Micrometer counters for success/failure/retry/validation.
 - Health endpoint with custom dependency checks and ``DEGRADED`` status.
 - Structured logs and Sentry integration.
- Security/Privacy:
 - PII masking for recipient in records and alerts.

1.8 Risks and Constraints

- ``recipientRaw`` is noted as encrypted in comments, but encryption is not enforced in current service logic.
- Elasticsearch indexing failures are logged and not retried; search can become eventually inconsistent with MongoDB.
- Main process catches broad exceptions and still acknowledges consumption flow; relies on DLQ to avoid loss.
